

LANGGRAPH IMPLEMENTATION

AI Chatbot Application Using LangGraph and Google Gemini API

2. Purpose of the Project

The purpose of this project is to develop an intelligent AI-based chatbot application that can interact with users in real time and provide automated responses using a Large Language Model (LLM). The project aims to demonstrate how conversational AI can be integrated into applications for automation, customer support, and interactive systems.

This project helps in understanding the use of modern AI frameworks, API integration, and state-based workflow management for building scalable AI-powered applications.

3. Project Description

This project is a console-based AI chatbot application developed using Python, LangGraph, LangChain, and Google Gemini API. The system accepts user input, sends it to the Gemini LLM model, processes the response, and displays intelligent replies back to the user.

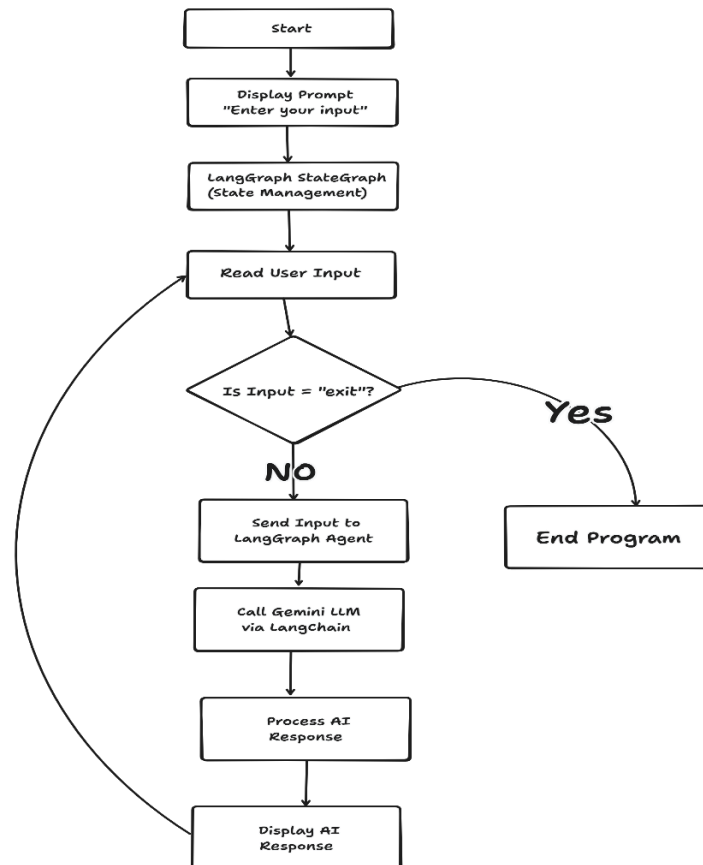
The chatbot runs in a continuous loop until the user types "exit". The application uses a graph-based workflow to manage the interaction state, making the system modular and extensible for future enhancements.

Target users include developers, students, and organizations who want to understand or implement conversational AI systems for learning, automation, or support use cases.

4. Workflow / Process Flow

User enters a message in the console.

The input is passed to the LangGraph state graph.
The agent node sends the input to the Google Gemini LLM.
The LLM processes the request and generates a response.
The response is extracted and formatted.
The system displays the AI response to the user.
The process repeats until the user types "exit".



5. Technology Stack

Programming Language: Python

AI Model: Google Gemini (gemini-3-flash-preview)

Frameworks/Libraries:

LangGraph

LangChain

langchain_google_genai

Environment Management: OS Environment Variables

Development Tools: VS Code /

Platform: Windows

6. System Design

6.1 ARCHITECTURE (TEXTUAL REPRESENTATION)

User (Console Input)

→ Python Application

→ LangGraph State Manager

→ LangChain Interface

→ Google Gemini LLM API

→ Response Processing

→ Console Output to User

6.2 MAJOR MODULES

Input Handling Module: Accepts user input from console.

Agent Module: Sends user input to the LLM and receives responses.

State Management Module: Uses LangGraph to manage application flow.

Response Processing Module: Extracts and formats text from the LLM response.

Output Module: Displays AI responses to the user.

6.3 DATA FLOW

User input is captured in the console.

Input is passed as state to the agent node.

Agent node sends request to Gemini API.

Gemini API returns AI-generated response.

Response is processed and stored in agent_response.

Final response is displayed back to the user.